

Software ontwikkeling
en professionele vaardigheden

2de bachelorjaar informatica

Robot Simulator: Analyseverslag

Nette Beckers (2159098)

Begeleider
Arno Verstraete

Coördinerend verantwoordelijke
prof. dr. Wim Lamotte

Jaar 2022-2023

Inhoudsopgave

1 Libraries en frameworks **2**
- 1.1 MySQL . . . 2
- 1.2 Blockly . . . 2
- 1.3 Node.js . . . 2
- 1.4 React.js . . . 2

2 Debugsysteem **3**
- 2.1 JS-interpreter . . . 3
- 2.2 Eigen implementatie . . . 3

3 Feedbacksysteem **3**

4 Datastructuren **4**
- 4.1 Robot structuur . . . 4
- 4.2 Level structuur . . . 4
- 4.3 Database . . . 4
  - 4.3.1 User . . . 4
  - 4.3.2 UserLevel . . . 5

5 Planning **5**
- 5.1 Mijlpalen . . . 5

1 Libraries en frameworks

De webapplicatie van de robot simulator zal opgebouwd worden met volgende libraries en frameworks.

1.1 MySQL

Voor het databasesysteem zal er gebruik gemaakt worden van MySQL. Deze geeft de mogelijkheid om een JSON-bestand op te slaan.

1.2 Blockly

Blockly voor web biedt veel mogelijkheden. Degene die in deze applicatie gebruikt zullen worden zijn:

- Blokken visueel aanmaken en uitvoeren
- Costum blokken maken
- Workspace aanmaken
- Blok explorer aanmaken
- Blokken opslaan en terug inladen
- Thema maken

1.3 Node.js

Voor de backend zal er gebruik worden gemaakt van Node.js. Deze laat toe om een gemakkelijke connectie met de database te maken en JavaScript aan de serverside uit te voeren.

1.4 React.js

Om de frontend op te bouwen en een overzicht te houden, zal er gebruik worden gemaakt van React.js. Deze laat toe om een eigen virtuele DOM te bouwen en zonder te refreshen deze te wijzigen.

2 Debugsysteem

2.1 JS-interpreter

Dit is een project geschreven voor Blockly, deze heeft het debugsysteem inzich. Hierbij kan er stap per stap door de code gegaan worden, herbeginnen, enzovoort. Deze heb ik eens doorgenomen om de werking te bekijken van Blockly met een debugsysteem. [3]

2.2 Eigen implementatie

Er zijn veel mogelijkheden, binnen Blockly is er een mogelijkheid om het blok die uitgevoerd wordt te highlighten. Als de code uitgevoerd wordt kan deze toegepast worden, maar ook bij het stap voor stap debuggen.

```
1 function highlightBlock(id) {
2   workspace.highlightBlock(id);
3 }
4
5 javascriptGenerator.STATEMENT_PREFIX = 'highlightBlock(%1);\n';
6 javascriptGenerator.addReservedWords('highlightBlock');
```

Listing 1: Highlight Blocks [1]

Voor het stap voor stap debugsysteem wordt de code van de workspace binnen Blockly opgevraagd en bijgehouden in een lijst. Met een index wordt er bijgehouden welke blok er wordt uitgevoerd. Bij elke stap vooruit of terug gaat dan de index wijzigen. Met deze index kunnen we dan het gewenste blok gaan uitvoeren en highlighten.

In enkele projecten met Blockly kwam het toevoegen van breakpoints ook terug. Dit kan worden toegevoegd met hetzelfde principe, een lijst met breakpoints die dan de indexen voorstellen van de lijst met blokken.

3 Feedbacksysteem

Blockly biedt de mogelijkheid om infinite loops te detecteren. Door deze functie zal per loop op een oneindige loop getest worden.

```
1 window.LoopTrap = 1000;
2 javascriptGenerator.INFINITE_LOOP_TRAP = 'if(--window.LoopTrap == 0) throw "Infinite
      loop.";\n';
3 var code = javascriptGenerator.workspaceToCode(workspace);
```

Listing 2: Detect Infinite Loops [1]

Verder is het belangrijk om qua feedback kunnen aan te geven waar het fout loopt. Zoals het rijden tegen randen en het fout navigeren, daarbuiten per level kunnen er ook aandachtspunten worden opgesteld. Deze kunnen dan geëvalueerd worden door het feedbacksysteem en de fouten weergeven. Bijvoorbeeld dat er een van de stappen die ze moeten voltooien is overgeslagen.

Het feedbacksysteem zal als volgt te werk gaan. De user kan opvragen om hun fouten weer te geven en dan wordt er een overzicht gegeven van waar het fout loopt. Ook zal er tijdens het debuggen de juiste foutmeldingen gegeven worden. Als er een fout toegewezen kan worden aan een bepaald blok zal dit bij het blok zichtbaar worden en anders in een apart canvas.

4 Datastructuren

Er zijn enkele belangrijke datastructuren nodig om de webapplicatie efficiënt op te bouwen. Verder is er nog een overkoepelende Game structuur die alles in goede banen zal leiden en linken met elkaar.

4.1 Robot structuur

Voor de robot zal er een structuur zijn die alle eigenschappen van de robot inhoud en de acties die hij kan uitvoeren. Deze structuur kan gebruikt worden om de custom blokken aan te maken. Zoals bijvoorbeeld het rijden, zijn sensors en berekeningen maken. In deze structuur wordt ook de toestand van de robot bijgehouden.

4.2 Level structuur

Om makkelijk en snel levels op te bouwen wordt er een structuur gemaakt dat een level voorstelt. Belangrijke zaken dat we moeten weten zijn:

- moeilijkheidsgraad van het level en nummer van het level
- doeleinden in een korte beschrijving, wat wordt er met dit level aangeleerd
- missies die voltooid moeten worden (aan de hand van een voorop gestelde lijst)
- beginpositie van de robot
- attributen en hun posities
- blokken die nodig zijn om het level te voltooien

Het level maakt gebruik van attributen, deze zullen nog in een eigen structuur worden opgeslagen. Waarin de staat bijgehouden wordt en de acties die er mee uitgevoerd kunnen worden. De attributen kunnen enkel via het level gebruikt of gewijzigd worden.

4.3 Database

De kinderen zullen hun eigen account hebben, dit vraagt om een database. Hierin worden de persoonlijke gegevens bijgehouden en de vorderingen per level. Blockly heeft een ingebouwde implementatie om blokken in een level op te slaan en in te laden. Dit gebeurt aan de hand van JSON-bestanden.

```
{
  "blocks": { ... },
  "workspaceComments": [ // Provided by workspace-comments serializer
    {
      "x": 239,
      "y": 31,
      "text": "Add 2 + 2"
    },
    // etc...
  ]
}
```

Listing 3: JSON output Blockly [2]

4.3.1 User

Er is een tabel voor de users die de inloggegevens bijhoudt.

- userID (uniek)
- mailadres
- hashed wachtwoord

4.3.2 UserLevel

Er is een tabel van levels met users. Hierin wordt het JSON-bestand met de code opgeslagen dat een bepaalde user bij een bepaald level heeft gemaakt.

- userID
- levelID (unieke combinatie van moeilijkheidsgraad en nummer van level)
- JSON-bestand

UserID met levelID geeft een unieke combinatie.

5 Planning

Er zijn enkele belangrijke mijlpalen opgesteld. Het visuele aspect staat er niet in vermeld, dit zal geleidelijk aan tot stand komen met de mijlpalen. Voor de robot zal er gestart worden met een vierkantje en naar mate dat er tijd over is, zal ik deze nog mooier vorm geven, net zoals alle andere attributen. Blockly heeft een handig ingebouwd systeem om een eigen thema toe toevoegen. Voor het algemeen design zal het sober qua vormgeving zijn, maar met enkele kleuren snel opgefleurd worden.

5.1 Mijlpalen

- **27 juli 2023** Blockly integreren
- **1 augustus 2023** Debugsysteem
- **4 augustus 2023** Robot en level structuur en custom blokken
- **8 augustus 2023** Feedbacksysteem
- **11 augustus 2023** Users en database integreren
- **13 augustus 2023** Levels maken en alles afwerken

Referenties

[1] Google Developers. Generating and running javascript. https://developers.google.com/blockly/guides/app-integration/running-javascript, 2022. Accessed in July, 2023.

[2] Google Developers. Saving and loading. https://developers.google.com/blockly/guides/configure/web/serialization, 2022. Accessed in July, 2023.

[3] Neil Fraser. Js-interpreter. https://github.com/NeilFraser/JS-Interpreter/tree/master, 2023. Accessed in July, 2023.